

Unified Dark Mode and UI Consistency for Django's Issue Tracker

Google Summer of Code 2026 Proposal

Organisation: Django Software Foundation

Proposal Category: Own Project

Project Title: Unified Dark Mode and UI Consistency for Django's Issue Tracker

Project Size: 175 hours

Difficulty: Medium

Forum: [Dark mode inconsistency between main site and Issues section](#)

Potential Mentor: Sarah Abderemane

Applicant: Keha Chandrakar

Email: Kehachandrakar07@gmail.com

GitHub: <https://github.com/kehach07>

University: Indian Institute of Technology Madras — BS in Data Science

Timezone: IST (UTC +5:30)

Abstract

This proposal is for a 175-hour Google Summer of Code 2026 project to improve Django's issue tracker by adding dark mode support and making its interface more consistent with the main Django website.

Right now, the main Django website supports dark mode, but the Issues section at code.djangoproject.com uses a separate Trac-based interface with different styling. This creates an inconsistent experience for users moving between the two sites.

The goal of this project is to add dark mode support to the issue tracker, improve the look and readability of important pages, and build the changes in a clean and maintainable way. It will also include documentation so future contributors can continue improving the interface more easily.

The final result will be a more consistent, accessible, and user-friendly experience across Django's website and issue tracker.

Related forum discussion: [Dark mode inconsistency between main site and Issues section](#)

Problem Statement

Django's main website and its issue tracker do not currently provide a consistent visual experience. The main website supports dark mode, but the Issues section at code.djangoproject.com uses a separate Trac-based interface with different styling. As a result, when users move from the main site to the issue tracker, the theme and overall look change suddenly.

This inconsistency affects both usability and contributor experience. The issue tracker is an important part of Django's public infrastructure, used for browsing tickets, reading reports, and contributing to discussions. A more consistent and accessible interface would make this experience smoother, more comfortable, and more usable for all users, regardless of whether they use dark mode or not.

Since the issue tracker is maintained separately, this improvement needs dedicated work in its own templates and styles. This project aims to address that gap in a practical and maintainable way.

Project Goals

The main goals of this project are:

1. Add dark mode support to Django's issue tracker.
2. Improve visual consistency between the main Django website and the Issues section.
3. Improve readability, contrast, and accessibility across important pages.
4. Organize the styling in a maintainable way for future contributors.
5. Document the implementation and validation process.

Deliverables

The expected deliverables for this project are:

1. Dark mode support for the main pages of code.djangoproject.com, following the same behavior as djangoproject.com.
2. Updated styling for key shared UI components such as navigation, forms, tables, links, and notices.
3. Improved visual consistency between Django's main website and the issue tracker where practical.
4. Accessibility improvements, including better contrast and clearer focus states.
5. A maintainable theme structure that reduces scattered hardcoded styling.
6. Documentation explaining the theme implementation and how future contributors can extend it.
7. A testing and validation checklist for contributors and maintainers to review UI consistency and accessibility across important pages.

Proposed Approach

This project will focus on improving the Trac-based issue tracker used at code.djangoproject.com. Since it is maintained separately from Django's main website, dark mode support and UI consistency need to be implemented directly in its own templates and styles.

The work will begin with a review of the current interface to identify the main pages and shared components that need updates. This includes ticket list pages, ticket detail pages, wiki pages, reports, navigation, tables, forms, links, and notices.

After this review, I plan to improve the styling structure of the issue tracker in a clean and maintainable way. Instead of adding isolated CSS fixes, the approach will focus on reusing styles from [django project .com](https://code.djangoproject.com) wherever possible and adapting them to the Trac-based interface. This will help maintain consistency while keeping the implementation practical and maintainable.

Dark mode support will be implemented step by step, starting with the main layout and shared components such as the header, navigation, tables, forms, links, and notices. These components will be updated first to ensure a consistent base across all pages.

After that, dark mode will be applied to key pages including ticket listing pages, ticket detail pages, wiki pages, reports, and search pages. For each page, I will review background colors, text contrast, borders, and interactive elements to ensure readability and usability.

Along with visual consistency, the work will focus on accessibility improvements such as maintaining readable contrast levels, improving focus states for keyboard navigation, and ensuring usability in both light and dark themes.

The project will also include documentation and a simple validation process to help future contributors understand the styling structure and review UI changes more easily.

Technical Plan

The goal of this project is to introduce dark mode support to Django's issue tracker (code.djangoproject.com), which is powered by Trac and uses its own styling layer. The implementation will focus on providing a maintainable theme system while keeping compatibility with the existing Trac infrastructure.

Development and testing will be performed using a local Trac instance created from the [djangoproject.com](https://code.djangoproject.com) repository, allowing CSS and template changes to be validated before submitting patches.

1. Local Development Environment

The project will be developed and tested using the repository setup instructions provided in the [djangoproject.com](https://code.djangoproject.com) codebase.

The workflow will be:

1. Create and activate a virtual environment.
2. Install development dependencies.
3. Initialize a local Trac environment.
4. Import the Django issue tracker database schema.
5. Run the local Trac server for testing.

This provides a working local instance of the issue tracker where UI changes can be tested across different pages such as:

- ticket lists
- ticket detail pages
- wiki pages
- timeline and reports

Outcome:

A fully functional local Trac environment used for development and testing.

2. Analysis of the Existing Trac Interface

Before implementing dark mode, the structure of the Trac interface will be analyzed using browser developer tools and the repository's CSS and template files.

Key components to analyze include:

Layout containers

(body, #content, #main)

Navigation components

(main navigation bar with WIKI, TIMELINE, ROADMAP, VIEW TICKETS, SEARCH)

Content components

(tables, forms, ticket lists, wiki pages)

Sidebar elements and auxiliary UI sections.

Outcome:

A clear mapping of Trac UI components and the selectors required to implement theme styling.

3. Dark Mode Styling Architecture

A separate stylesheet will be introduced:

dark-mode.css

This stylesheet will contain overrides for Trac's default styles.

To maintain compatibility and reduce risk:

- existing styles such as `trachacks.css` will not be modified directly
- the new stylesheet will be loaded after the existing styles
- selectors will target specific UI components to avoid unintended overrides

Example structure:

```
html[data-theme="dark"] body { ... }
```

```
html[data-theme="dark"] #content { ... }
```

```
html[data-theme="dark"] .listing { ... }
```

Outcome:

A maintainable styling layer that allows dark mode without altering existing Trac CSS files.

4. Theme Toggle Implementation

A theme toggle will be introduced in the issue tracker navigation bar, allowing users to switch between:

- Light mode
- Dark mode
- System preference

Implementation approach:

A theme attribute will be applied to the root element:

```
<html data-theme="dark">
```

JavaScript will be used to:

- switch themes dynamically
- store user preference using `localStorage`
- fall back to system preference using `prefers-color-scheme`

Example logic:

```
localStorage.setItem("theme", "dark")
```

The toggle will be integrated into the Trac navigation layout without disrupting existing navigation elements.

Outcome:

A persistent theme toggle that allows users to switch modes easily.

5. Incremental Implementation

The styling implementation will be performed incrementally to ensure stability.

Phase 1 — Base Layout Styling

Apply dark background and text colors to the main layout elements such as body, content containers, and page structure.

Phase 2 — Navigation Styling

Update the navigation bar and tab navigation elements used across Trac pages.

This includes sections such as:

- WIKI
- TIMELINE
- ROADMAP
- VIEW TICKETS
- SEARCH

Hover states and active states will be adjusted to ensure readability in dark mode.

Phase 3 — Tables and Ticket Lists

Update `.listing` tables used in ticket listings and reports.

Changes will include:

- header styling
- row backgrounds
- borders and separators

Phase 4 — Forms and Inputs

Style input fields, search boxes, buttons, and other form components to ensure visibility and usability in dark mode.

Phase 5 — Secondary UI Components

Update sidebar elements, wiki formatting blocks, and secondary UI components to maintain visual consistency.

Phase 6 — Edge Cases

Handle additional components such as:

- wiki pages
- code blocks
- pagination
- notifications

Outcome:

A consistent dark mode implementation across the main interface components.

6. Testing and Validation

Testing will be performed using the local Trac instance.

Changes will be validated across different sections of the issue tracker including:

- ticket lists
- ticket detail pages
- wiki pages
- reports and timeline

Validation will focus on:

- visual consistency
- text readability
- accessibility (contrast and focus states)

Outcome:

A stable and readable dark mode experience across the interface.

7. Integration with Trac

The dark mode stylesheet will be integrated using Trac's template system.

Possible integration points include:

- `site_head.html`
- custom template overrides

The stylesheet will be loaded after existing styles so it can override default styling without modifying core Trac assets.

Outcome:

A clean integration that maintains compatibility with existing Trac infrastructure.

8. Documentation

The implementation will include documentation describing:

- the structure of the dark mode stylesheet
- key selectors used for overrides
- the theme toggle implementation
- guidelines for extending theme support in the future

This documentation will help maintainers and contributors understand and extend the implementation.

Project Timeline

Community Bonding Period

During the community bonding period, I will focus on preparing the development environment and confirming the implementation plan with mentors.

This phase will include:

- Setting up the local development environment using the **django**project.com repository instructions.
- Running a **local Trac instance** to understand the structure of the Django issue tracker.
- Reviewing the styling system used by the tracker, including **trachacks.css** and template overrides.
- Identifying the main interface components used across the issue tracker (navigation, ticket tables, wiki pages, forms).
- Defining the final dark mode design and confirming the theme toggle behavior (light, dark, system mode) with mentors.

Outcome:

A fully working local development environment and a confirmed implementation roadmap.

Phase 1 (Week 1–2): Base Theme Infrastructure

During this phase I will implement the core theme architecture.

Work will include:

- Introducing a **dark-mode.css** stylesheet.
- Implementing the **theme system using a data-theme attribute** on the HTML root element.
- Defining base theme variables and color palette.
- Applying dark mode styling to the main layout structure.

Key areas covered:

- page background and content containers
- base typography and link styling
- layout containers used across Trac pages

Milestone:

The issue tracker layout renders correctly in dark mode with consistent base styling.

Phase 2 (Week 3): Navigation and Global UI Components

This phase focuses on navigation elements and globally shared interface components.

Work will include styling:

- the main navigation bar
- tab navigation elements (Wiki, Timeline, Roadmap, View Tickets, Search)
- active and hover states
- header elements and page titles

Milestone:

Navigation and global interface elements are visually consistent and usable in dark mode.

Phase 3 (Week 4–5): Ticket Views and Data Tables

This phase will focus on pages where contributors spend most of their time.

Work will include improving styling for:

- ticket list tables (`.listing`)
- ticket detail views
- report tables
- timeline views

Special attention will be given to readability and contrast.

Milestone (Midterm Evaluation):

Dark mode is fully functional across ticket lists, reports, and timeline pages.

Phase 4 (Week 6): Forms, Inputs, and Theme Toggle

This phase introduces user interaction improvements and the theme switcher.

Work will include styling:

- search fields
- form inputs
- buttons and controls

The **theme toggle** will also be implemented in the navigation bar.

Supported modes:

- Light mode
- Dark mode
- System preference (based on `prefers-color-scheme`)

User preference will be stored using `localStorage`.

Milestone:

Theme switching works correctly and persists across page loads.

Phase 5 (Week 7): Secondary Interface Components

This phase focuses on remaining UI sections to ensure visual consistency.

Work will include styling:

- sidebar components
- wiki formatting blocks
- code blocks
- additional navigation elements

Milestone:

All secondary UI components are visually consistent with the dark theme.

Phase 6 (Week 8): Edge Cases and Refinement

This phase focuses on final adjustments and polishing.

Tasks include:

- handling less common UI components
- fixing layout inconsistencies
- improving color contrast where necessary
- refining hover and focus states

Milestone:

Dark mode works consistently across all major pages and UI components.

Phase 7 (Week 9): Testing and Documentation

The final phase focuses on validation and documentation.

Testing will be performed on the local Trac instance across:

- ticket pages
- wiki pages
- reports and timeline

Documentation will include:

- structure of the dark mode stylesheet
- explanation of theme toggle behavior
- guidelines for extending theme styling

Milestone:

Stable implementation with clear documentation ready for review.

Final Deliverable

The final outcome of the project will include:

- a maintainable **dark mode implementation** for the Django issue tracker
- a structured stylesheet (`dark-mode.css`)
- a **theme toggle supporting light, dark, and system modes**
- consistent styling across ticket pages, wiki pages, and reports
- documentation to support future improvements

Motivation and Why I Am a Good Fit

I am interested in this project because it improves a real part of Django's contributor experience. The issue tracker is used often, and making it more consistent, accessible, and easier to use is meaningful work for the community.

I believe I am a good fit for this project because I have been working with Django for the past three years, including during my internship, where I worked on real projects and communicated with international clients. This helped me build strong debugging, communication, and delivery skills.

My recent open-source contributions have focused on UI fixes and improvements in Django-related projects, especially around the `django` project website (detailed below in the contributions section). Through this work, I have gained practical experience working with interface issues, styling consistency, and dark mode related improvements.

This project matches both my experience and my interest. I am comfortable working with existing codebases, fixing issues carefully, and communicating regularly during development.

Open Source Contributions

GitHub: <https://github.com/kehach07>

I have actively contributed to Django-related projects, particularly within the Django ecosystem. Through these contributions, I have gained practical experience with Django's contribution workflow, code review process, and the level of quality expected in open-source development.

A significant portion of my recent work has focused on UI improvements, frontend fixes, and website-related changes, especially in djangoproject.com and other Django-associated repositories. These contributions closely align with this proposal, as they involve interface consistency, usability improvements, and working with contributor-facing platforms.

Selected Contributions

Contribution	Repository	Status	Relevance
Vendor Leaflet JS and CSS locally (replace CDN)	django/birthday20	Merged	Improved frontend reliability and asset handling
Add blue label and background for "Workshop" events	django/birthday20	Merged	UI enhancement and visual clarity
Add missing translation strings (header, footer, menu)	django/djangoproject.com	Pending	Improved user-facing content and structure
Fix and improvements (PR #2502)	django/djangoproject.com	Reviewed	Contribution to Django website codebase
Multiple UI and content fixes (see PRs below)	django/djangoproject.com	Open	Ongoing involvement in website improvements

Pull Requests and Issues

The following are selected contributions relevant to this proposal:

Merged

- [django/birthday20 PR #114](#)
- [django/birthday20 PR #112](#)

Reviewed / In Progress

- [django/djangoproject.com PR #2441](#)
- [django/djangoproject.com PR #2502](#)

Other Contributions

- [django/djangoproject.com PR #2496](#)
- [django/djangoproject.com PR #2426](#)
- [django/djangoproject.com Issue #2474](#)
- [django/djangoproject.com Issue #2500](#) — Pending

What These Contributions Have Helped Me Build

Through these contributions, I have:

- become comfortable working with Django-related repositories and existing codebases
- improved my understanding of review-driven open-source development
- gained hands-on experience with UI-focused fixes and website improvements
- learned to iterate based on maintainer feedback and code reviews
- developed familiarity with changes that affect user-facing parts of Django project

Why This Matters for This Project

These contributions are directly relevant to this proposal, as my work has already focused on UI fixes, frontend consistency, and improvements to contributor-facing Django platforms. I have also worked on styling-related changes and have practical experience with dark mode and interface consistency, making this project a strong match for both my skills and my recent open-source involvement.

Availability and Commitments During GSoC

During the GSoC coding period, I plan to dedicate approximately **30–35 hours per week** to the project and stay focused on completing the major milestones on schedule, particularly before the mid-term evaluation.

My academic workload during the summer will be manageable, allowing me to maintain consistent progress throughout the coding period. From **10 June to 14 June**, I will have slightly limited availability due to academic commitments, but I will still continue lighter tasks such as planning, reviewing progress, documentation, and communication with mentors.

Throughout the project, I will maintain regular communication with mentors, share progress updates, and ensure that development stays aligned with project goals and community expectations.

Communication Plan

I will maintain regular communication with mentors throughout the project and share progress updates consistently. I am comfortable discussing implementation details, asking questions when needed, and responding to feedback during review. I also plan to stay active in the Django community through the Django Forum, relevant GitHub discussions and pull requests, and project-related communication channels to ensure development remains aligned with mentor and community expectations.

Conclusion

This project aims to improve Django's issue tracker by adding dark mode support and making its interface more consistent with the main Django website. By focusing on usability, accessibility, and maintainable implementation, the project will provide a practical improvement to an important contributor-facing part of Django's documentation and resources. I would be excited to contribute this work to Django through GSoC 2026.